

Manual Técnico — MediLogic

Curso: Inteligencia Artificial 1 — Universidad San Carlos de Guatemala **Proyecto:** MediLogic — Entrega No. 2, Proyecto Final

Este documento es la versión final del Manual Técnico. Reemplaza el borrador de la Entrega No. 1 (ver historial de `git Log` de este archivo si se necesita comparar).

1. Resumen del proyecto

MediLogic es un sistema experto de diagnóstico médico preliminar. Un motor de inferencia escrito íntegramente en Prolog (`src/prolog/knowledge_base.pl`) recibe los síntomas (con su severidad), alergias y enfermedades crónicas de un paciente y calcula, para cada enfermedad conocida, un porcentaje de afinidad, un nivel de urgencia y un medicamento seguro sugerido. Un módulo administrativo (protegido por autenticación) permite mantener esa base de conocimiento sin editar el archivo `.pl` a mano, incluyendo un robot (RPA) que agiliza el alta masiva de enfermedades, tanto en modo directo como operando la interfaz con PyAutoGUI.

2. Arquitectura del sistema

Arquitectura cliente-servidor: un cliente web (navegador) consume una aplicación Flask que actúa como orquestador entre la interfaz y el motor lógico Prolog (vía `pyswip`). Toda la lógica de decisión (afinidad, contraindicaciones, urgencia) vive exclusivamente en `knowledge_base.pl` ; Python nunca la reimplementa, solo la invoca y transforma el resultado para la interfaz web (interpretación documentada de la sección 4.3 del enunciado: "*la lógica de Prolog exclusivamente debe ser en Python*" = Python actúa solo como puente).

Ver los diagramas completos (arquitectura general, secuencia de diagnóstico y secuencia de actualización administrativa con RPA) en [docs/diagramas/arquitectura.md](#).

Componentes principales:

Componente	Ubicación	Responsabilidad
Interfaz web	<code>src/backend/templates/</code> , <code>src/backend/static/</code>	Formularios y resultados (paciente/administrador)
Rutas Flask	<code>src/backend/routes/</code>	Recibir peticiones HTTP y orquestar la respuesta
Puente Prolog	<code>src/backend/prolog_engine.py</code>	Consultar/recargar <code>knowledge_base.pl</code> vía <code>pyswip</code>
Persistencia administrativa	<code>src/backend/knowledge_store.py</code>	CRUD → reescribe únicamente las secciones de hechos del <code>.pl</code>
Base de conocimiento	<code>src/prolog/knowledge_base.pl</code>	Toda la lógica de diagnóstico (hechos y reglas)
Informe PDF	<code>src/backend/pdf_report.py</code>	Genera el informe descargable (ReportLab)
RPA — modo rápido	<code>src/rpa/admin_rpa.py</code>	Alta masiva desde JSON, llamando directo a <code>KnowledgeStore</code>

RPA — modo visual	src/rpa/gui_automation.py	La misma alta masiva, operando la interfaz real con PyAutoGUI
-------------------	---------------------------	---

3. Herramientas y tecnologías utilizadas

- **Python 3 + Flask** — capa web ligera, ver justificación en docs/decisiones_tecnicas.md §1.
- **pyswip (sobre SWI-Prolog)** — integración con el motor lógico real, ver §2.
- **PyAutoGUI + pyperclip** — automatización RPA del lado del backend, ver §3 y §14.
- **ReportLab** — generación del informe de diagnóstico en PDF, ver §4.
- **pytest** — pruebas automatizadas del backend (17 pruebas, ver sección 8 de este manual).

4. Estructura del archivo knowledge_base.pl

El archivo está organizado en 12 secciones numeradas y comentadas:

1. Catálogo de síntomas (sintoma/1)
2. Catálogo de medicamentos (medicamento/1)
3. Enfermedades (enfermedad/4 : nombre, descripción, sistema del cuerpo, tipo)
4. Relación enfermedad-síntoma (enfermedad_sintoma/2)
5. Relación enfermedad-medicamento que la trata (medicamento_para/2)
6. Contraindicaciones fármaco-patología (contraindication_enfermedad/2)
7. Catálogos derivados: `alergia/1` (deriva de `medicamento/1`) y `enfermedad_cronica/1` (deriva de `enfermedad/4` filtrando `tipo = cronico`)
8. Ponderación de severidad (peso_severidad/2) y cálculo de afinidad (afinidad/3)
9. Nivel de urgencia (nivel_urgencia/3)
10. Seguridad de medicamentos: contraindicación por alergia y por enfermedad crónica (medicamento_seguro/3 , medicamento_sugerido/4)
11. Diagnóstico e informe completo (diagnosticar/2 , informe/4)
12. Consultas de ejemplo (comentadas)

Las secciones 1–6 (hechos) están delimitadas con marcadores `% === AUTO:<seccion> START/END ===` : son las únicas que `knowledge_store.py` reescribe al guardar un cambio administrativo. Las reglas (secciones 7 en adelante) nunca se tocan de forma automática.

4.1 Justificación de las reglas principales

- **afinidad/3** implementa el criterio de la sección 4.2 del enunciado: puntos acumulados por síntomas coincidentes (ponderados por severidad: leve=1, moderado=2, severo=3) sobre el total máximo posible para esa enfermedad. El "máximo posible" se definió como todos los síntomas de la enfermedad en su grado más severo — ver docs/decisiones_tecnicas.md §8.
- **nivel_urgencia/3** traduce el porcentaje de afinidad a una de las tres recomendaciones textuales que exige el enunciado, con umbrales documentados en docs/decisiones_tecnicas.md §9 (≥60% alta, 20–59% media, <20% baja).
- **medicamento_sugerido/4** implementa los **dos niveles obligatorios de contraindicación** de la sección 4.2: descarta primero el medicamento contraindicado contra la enfermedad que se está tratando (`contraindication_enfermedad(Medicamento, Enfermedad)`), y luego contra las alergias/enfermedades crónicas propias del paciente (`medicamento_seguro/3`); si el primer candidato falla, prueba automáticamente el siguiente medicamento registrado para esa enfermedad. El nivel de alergia está verificado con datos reales (no solo teóricos) en docs/evaluacion_coherencia/ ; el nivel de enfermedad crónica está implementado y es simétrico al de alergia, pero el catálogo que trae el proyecto no genera ningún caso real que lo dispare — limitación documentada honestamente en

docs/evaluacion_coherencia/evaluacion_coherencia.md y en docs/decisiones_tecnicas.md §15.

- **diagnosticar/2** ordena los diagnósticos de mayor a menor afinidad usando `sort(2, @>=, ...)` en vez de `predsort/3`, para no perder enfermedades que empaten en porcentaje.

5. Consultas de ejemplo y su explicación

Consultas de referencia (sección 12 del `.pl`), verificadas con `swipl` y con `pyswip` (`scripts/demo_prolog.pl` / `scripts/demo_pyswip.py`) y con la aplicación Flask corriendo de extremo a extremo (`pytest`, 17/17 pruebas en verde):

1. `afinidad(hipertension, [dolor_cabeza-moderado, mareo-severo], P).` → `P = 56` (puntos $2+3=5$ sobre un máximo de 9).
2. `nivel_urgencia(56, Nivel, Recomendacion).` → `Nivel = media`, `Recomendacion = 'Posible automanejo'`.
3. `medicamento_sugerido(hipertension, [losartan], [], Medicamento).` → `Medicamento = enalapril` (el motor descarta losartán por la alergia y prueba el siguiente candidato registrado).
4. `diagnosticar([dolor_cabeza-moderado, mareo-severo, dolor_abdominal-leve], D).` → `[diagnostico(hipertension, 56), diagnostico(gastroenteritis_aguda, 8)]`.
5. `informe([dolor_cabeza-moderado, mareo-severo], [ibuprofeno], [], Informe).` → sugiere `losartan` (el paciente no es alérgico a él; ibuprofeno ni siquiera es candidato de tratamiento para hipertensión).

Ver además `docs/informe_prolog/plantilla_informe_prolog.md` (con capturas) y `docs/evaluacion_coherencia/` (tres casos clínicos completos ejecutados contra la aplicación real vía HTTP).

6. Flujo de interacción entre módulos

Ver [docs/diagramas/arquitectura.md](#): incluye el diagrama de secuencia del diagnóstico de un paciente y el de la actualización administrativa (con RPA).

7. Diseño de interfaz

Ver [docs/mockups/](#) — wireframes navegables de las 7 pantallas principales, con el flujo de navegación documentado en `docs/mockups/README.md`. La aplicación real (`src/backend/templates/`) sigue ese mismo diseño, ya implementada sobre Jinja2 y consumiendo las rutas Flask reales (no son mockups estáticos).

8. Pruebas automatizadas

`pytest` (17 pruebas, todas en verde) cubre:

- Arranque de la aplicación (`tests/test_app.py`).
- Autenticación y CRUD de enfermedades del módulo administrador, incluyendo exportación del `.pl` (`tests/test_admin.py`).
- Formulario de paciente, cálculo de afinidad, sustitución de medicamento por alergia, caso sin síntomas y descarga de PDF (`tests/test_diagnostico.py`).
- La lógica determinista del RPA visual (conteo de Tab/Espacio para llegar a cada casilla, selección de `<select>` y pegado desde el portapapeles) usando un doble de prueba en lugar de mover el mouse/teclado real (`tests/test_rpa_gui_logic.py`).
- El RPA modo rápido: carga exitosa con el formato original (actualiza sin duplicar), carga con el campo opcional `tratamiento_recomendado` (verifica que registre `medicamento_para/2` con datos reales) y

manejo controlado de un registro con campos faltantes (`tests/test_rpa.py`).

Todas las pruebas usan una **copia temporal** de `knowledge_base.pl` (`tmp_path` de `pytest`), nunca el archivo real del repositorio.

9. Configuración del robot RPA

El proyecto incluye dos formas de ejecutar el RPA de carga masiva de enfermedades (ver `docs/decisiones_tecnicas.md` §14 para la justificación completa de este diseño):

9.1 Modo rápido (botón "Ejecutar RPA" del panel administrativo)

`src/rpa/admin_rpa.py::ejecutar_carga` lee el JSON subido y llama directamente a `KnowledgeStore` (la misma capa que usa el formulario manual), sin depender de que haya un navegador con foco. Es el camino recomendado para uso diario del administrador. Acepta tanto el formato original (`EjemploRPA.json`) como una variante con el campo opcional `tratamiento_recomendado` (ver `Ejemplo Archivo RPA V2.json`); si el campo está presente, además registra qué medicamentos tratan cada enfermedad (`medicamento_para/2`), dando de alta en el catálogo cualquier medicamento nuevo que aparezca ahí — ver `docs/decisiones_tecnicas.md` §17.

9.2 Modo visual — automatización real con PyAutoGUI (para el video de evidencia)

`src/rpa/gui_automation.py` , ejecutado vía `python scripts/ejecutar_rpa_gui.py [ruta_al_json]` , controla el mouse/teclado del sistema operativo para operar la página web real del panel de administrador: inicia sesión, y para cada enfermedad del JSON llena el formulario "Nueva enfermedad" (creando antes, si hace falta, los síntomas/medicamentos que todavía no existan en el catálogo).

Cómo funciona, sin coordenadas de pantalla fijas:

Paso	Técnica usada	Por qué
Enfocar el primer campo de cada formulario	Atributo HTML <code>autofocus</code> en la plantilla	El navegador ya deja el cursor listo al cargar la página; no hace falta clic
Navegar entre páginas	Ctrl+L (barra de direcciones) + URL + Enter	Funciona igual en cualquier navegador/resolución/tamaño de ventana
Escribir texto (nombres, descripciones)	Copiar al portapapeles + Ctrl+V	<code>pyautogui.write()</code> no soporta con confiabilidad acentos/eñes del español
Elegir "Sistema del cuerpo" / "Tipo" (<select>)	Home + flecha abajo × N (N = índice del valor en la lista)	No depende del "type-ahead" del navegador
Marcar los checkboxes de síntomas/medicamentos	Tab × N + Espacio, con N calculado leyendo el catálogo real vía <code>PrologEngine</code> de solo lectura	El orden de los checkboxes en la plantilla es siempre alfabético; el robot cuenta exactamente cuántos Tab necesita en vez de adivinar una posición en píxeles

Prerrequisitos antes de grabar el video:

1. Servidor Flask corriendo (`python src/backend/app.py`) en una terminal.

2. Un navegador abierto y con foco real de ventana (PyAutoGUI mueve el mouse/teclado de *todo* el sistema operativo, no solo del navegador — no tocar el equipo mientras corre).
3. Deshabilitar el gestor de contraseñas del navegador (o usar una ventana de incógnito/InPrivate) para que no aparezca un popup de "¿guardar contraseña?" tras el login, que podría robarle el foco al robot.
4. Ejecutar `python scripts/ejecutar_rpa_gui.py EjemploRPA.json` (o el JSON que se quiera cargar) desde otra terminal. El script imprime una cuenta regresiva de 5 segundos antes de empezar a mover el mouse/teclado, tiempo para dejar el navegador al frente.
5. Para abortar en cualquier momento: llevar el mouse a cualquier esquina de la pantalla (PyAutoGUI FAILSAFE, activado por defecto) o `Ctrl+C` en la terminal.

Parámetros ajustables (clase `ConfiguracionGUI` en `gui_automation.py`): URL base, usuario/contraseña de administrador, y los tiempos de espera entre acciones (`espera_navegacion` , `espera_pegado`) — están en valores generosos por defecto para que el video quede legible; se pueden reducir si el equipo donde se graba es más rápido.

Al terminar, ambos modos del RPA generan una bitácora en texto plano con el mismo formato (`ReporteRPA.texto_bitacora()`), guardada en `instance/reportes/` y descargable desde el panel de administrador.

10. Decisiones técnicas y su justificación

Ver docs/decisiones_tecnicas.md — bitácora completa de las decisiones tomadas de forma autónoma durante el desarrollo (framework, librería Prolog, diseño del RPA visual, generación de PDF, interpretación de fórmulas, umbrales, ampliación del catálogo, y validación del motor lógico).